Plugin Architectures with Rust

an Analysis of Obstacles and Prospects

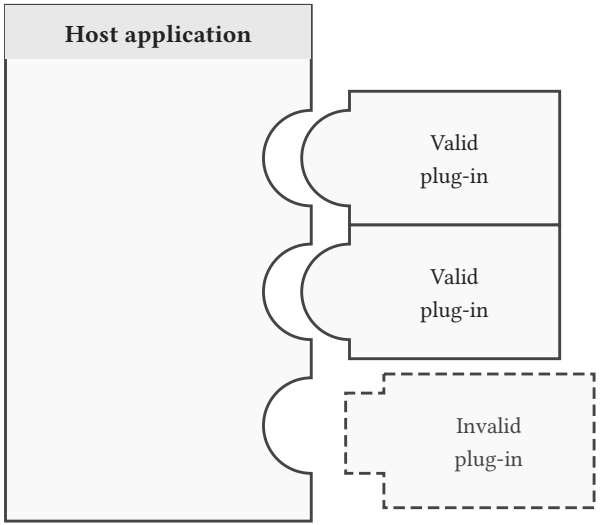
20.05.2026

Contents

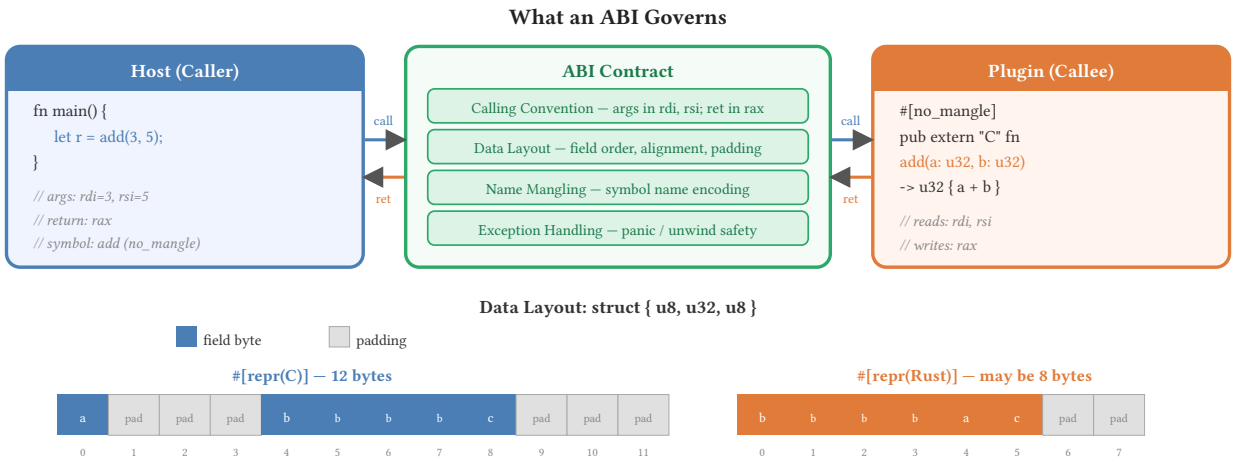
1 Introduction

Plugin Systems

- **Extensibility**: The ability to add new functionality post-deployment.
- **Modularity**: Plugins are isolated units that interact with the host through a well-defined interface.
- **Dynamic Loading**: Plugins are typically loaded at runtime rather than linked statically at compile time.



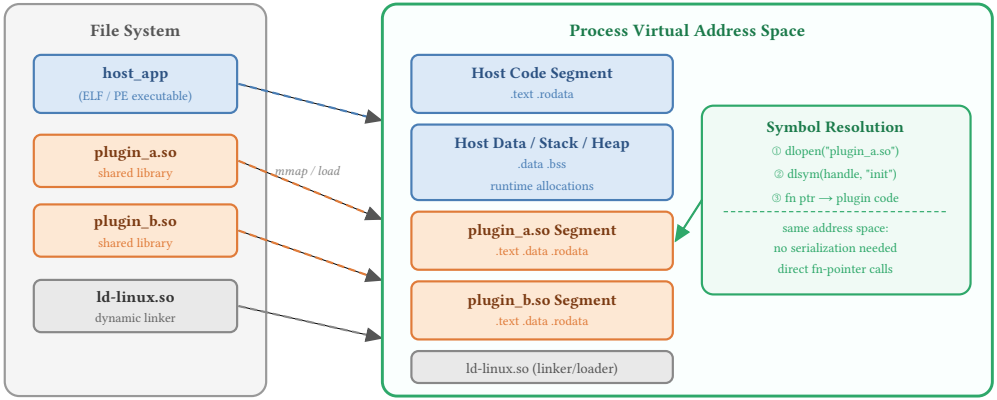
ABI


What an ABI Governs
Host (Caller)
fn main() {
let r = add(3, 5);
}
// args: rdi=3, rsi=5
// return: rax
// symbol: add (no_mangle)
call
ret
ABI Contract
Calling Convention — args in rdi, rsi; ret in rax
Data Layout — field order, alignment, padding
Name Mangling — symbol name encoding
Exception Handling — panic / unwind safety
call
ret
Plugin (Callee)
#[no_mangle]
pub extern "C" fn
add(a: u32, b: u32)
-> u32 { a + b }
// reads: rdi, rsi
// writes: rax
Data Layout: struct { u8, u32, u8 }
field byte
padding
#[repr(C)] — 12 bytes
a pad pad pad b b b b c pad pad pad
0 1 2 3 4 5 6 7 8 9 10 11
#[repr(Rust)] — may be 8 bytes
b b b b a c pad pad
0 1 2 3 4 5 6 7
The compiler may reorder fields — a C caller reading offset 8 would get garbage

Dynamic linking and loading

1.c.a Process of dynamic linking and loading

1) Discovery
2) Loading
3) Resolution
4) Execution



Dynamic linking and loading (ii)

1.c.b Example with Rust ABI

Using the `libloading` crate:

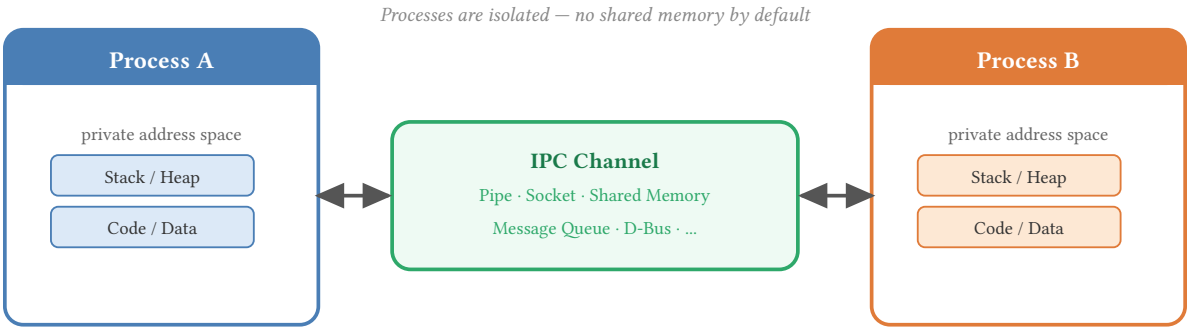
```
trait Fuser {
  fn fuse(&self, data: &[SensorData]) -> Result<SensorData, Box<dyn Error>>;
}
struct Plugin<T> where T: Fn() -> Box<dyn Fuser> + Copy{
  lib: Library,
  func: T,
}
fn new(path: &PathBuf, symbol_name: &[u8]) -> Result<Self, libloading::Error> {
  let lib = unsafe { Library::new(path) }?; // 1+2) Discovery and Loading
  let symbol: Symbol<T> = unsafe { lib.get(symbol_name) }?; // 3) Resolution
  let func: T = *symbol; // fn deref(&self) -> &T
  Ok(Plugin { lib, func })
}
```

Dynamic linking and loading (iii)

```
type FuseFunc = extern "Rust" fn() -> Box<dyn Fuser>;
let p: Plugin<FuseFunc> = Plugin::new(&plugin_path.into(),
"average_plugin".as_bytes()).unwrap();
a(&p);

fn a(plugin: &Plugin<FuseFunc>) {
  let fuser: Box<dyn Fuser> = plugin.get();
  let sd = [ SensorData::new(42.0, 0.01), ... ];
  let res = fuser.fuse(&sd); // 4) Execution
  println!("a) {:?}", res);
}
```

IPC



Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat.

IPC (ii)

Term
Definition

2 Conclusion

Performance

Crate	Perf. Diff	Development Complexity	Limitation	Interoperability
native		++	0	0
unstable_abi	16%	--	--	0
abi_stable	25%	0	0	+
stabby	9%	0	0	+
rust_bridge (JSON)	537%	+	0	++
rust_bridge (Binary)	25%	-	0	++

Table 1: Summary of results.